

خب ببین ما به تیم ۳ نفره ایم که دانشجوی مهندسی کامپیوتریم و پایتون و مبانی هوش مصنوعی و پردازش زبان طبیعی و معماری RAG رو تا حد پایه بلدیم

ولی اولین باره که پروژه نمیه حرفه ای قبول میکنیم و قراره که تازه مسیر و علم پیاده سازی پروژه رو حین اجرای پروژه یادگیریم

حالا پروژه ما چیه ؟

اسمش: "سیستم پرسش و پاسخ هوشمند آفلاین فارسی مبتنی بر PDF کاربر"

کاربرد: برای کمک به کاربرانی که اسناد محرمانه دارن، نیاز به پردازش دارن و از ابزارای آنلاین نمیتونن استفاده کنن به دلیل مسائل امنیتی

در واقع ما قراره که یه سیستمی بسازیم که کاربر به صورت آفلاین و روی سیستم خودش اسنادشو بارگذاری میکنه و سوالاتشو میپرسه و ما طبق اسناد خودش با ذکر منبع بهش پاسخ میدیم

و اگر پاسخش هم توی اسنادش نبود صراحتا بهش اعلام میکنیم

در ضمن توی چت های جدا میتونه گفتگو کنه و تاریخچه چت هاش نگه داری میشه

(بجورایی مثل چتبات نوت بوک کوچیک و آفلاین)

حالا تصمیماتی که تا الان برای این پروژه گرفتیم از این قراره:

۱. مسیر کلی پروژه:

رابط کاربری اسناد کاربر رو میگیره میده به ماژول استخراج متن

بعد متن استخراج شده میره به ماژول پیش پردازش و نرمال سازی فارسی

بعد قطعه قطعه میشه (chunking)

بعد قطعات فرستاده میشن به ماژول امبدر تا به بردار عددی تبدیل و تو پایگاه داده برداری ذخیره بشن

بعد از اون از طریق رابط کاربری به کاربر اعلام میشه که پردازش اسنادت تموم شده و میتونی سوالاتو بپرسی و سوال کاربر دریافت میشه و میره به ماژول پیش پردازش و از اونجا میره به ماژول امبدر تا به بردار عددی تبدیل بشه

بعد از اون ماژول بازیاب میاد شبیه ترین بردار های داخل پایگاه داده برداری به بردار سوال کاربر رو پیدا میکنه

و متن سوال کاربر به همراه متن چن تا قطعه برتر در قالب پرامپت مناسب فرستاده میشن برای مدل زبانی تا پاسخ کاربر رو تولید بشه و به همراه شماره صفحه و فایلی که پاسخ کاربر ازش تولید شده از طریق رابط کاربری به کاربر نمایش داده بشه

۲. برای پایگاه داده برداری قراره که از chroma DB استفاده کنیم

چون علاوه بر بردار ها و اندیشون ، متن خود چانک ها و متا دیتا رو هم نگه میداره و نیازی به پایگاه داده کمکی برای نگه داری متادیتا نداره

(برخلاف FAISS)

۳. برای روش بازیابی هم احتمالا از روش ترکیبی (واژگانی + معنایی) استفاده کنیم ولی بدلیل محدودیت زمانی که در پاسخگویی به کاربر داریم نمیتونیم از رتبه بندی مجدد استفاده کنیم (reranker)

*البته تصمیممون برای روش بازیابی ۹۰ درصد قطعی و ۱۰ درصد احتمال تغییر داره

۴. برای مدل امبدینگ به دلیل محدودیت های سخت افزاری قراره که برای فاز توسعه و کدنویسی اولیه از مدل سبک e5-small استفاده کنیم ولی برای فاز یکپارچه سازی و تست نهایی پروژه قراره که بریم روی کولب یا یه سیستم قوی محلی و تو اون مرحله بسته به سخت افزاری که در اختیار داریم از مدل های سنگین تر مثل کوئن ۸ میلیارد استفاده کنیم

بخاطر همین باید موقع توسعه کد رو طوری بنویسیم که موقع تغییر مدل کمترین تغییر تو کد نیاز باشه

(مثلا تا الان تصمیم گرفتیم که فایل save-models رو برای داندلود و ذخیره مدل ها به صورت خودکار داشته باشیم که با اجرا کردنش در صورت نبود مدل تو پوشه ذخیره مدل ها اونا رو بیار داندلود و ذخیره کنه

یا فایل config.py رو داریم که قراره اسم و مشخصات و ابعاد وکتور مدل و.. توش بنویسیم و باقی فایل ها موقع فراخوانی و استفاده از مدل ، مشخصات و اسمش رو از این فایل بخونن که موقع تغییر مدل ها فقط یه کلید داخل فایل کانفیگ رو نیاز باشه تغییر بدیم

یا فایل reset-DB.py برای اینه که بعد از تغییر مدل ، کل پایگاه داده پاک بشه و مجدد با مدل امبدینگ جدید ساخته بشه که مشکل ناسازگاری ابعاد بردار ها پیش نیاد

(...و)

۵. برای مدل زبانی هم مجددا بخاطر محدودیت سخت افزاری سیستم های خودمون قراره که تو فاز توسعه اولیه از مدل سبک کوئن ۳:۵۰۲ B (احتمالا) استفاده کنیم ولی بعد که رفتیم روی کولب یا سیستم قوی محلی با توجه به مشخصات اون سخت افزار و میزان رمی که مدل امبدینگ مصرف میکنه یه مدل کمی سنگین تر (مثل دیپ سیک ۱۴ میلیارد) استفاده میکنیم

۶. ما یسری محدودیت ها داریم

مثلا اینکه زمان پاسخگویی به سوال کاربر باید حداکثر ۳ ثانیه باشه

و برای فاز توسعه که به شدت محدودیت سخت افزاری داریم

و برای فاز تست و اجرای نهایی هم یکم دستمون تو سخت افزار باز تره ولی هنوز مطمئن نیستیم که چه سیستمی قراره داشته باشیم

بخاطر همین هنوز نمیتونیم تصمیم قطعه برای اون موقع بگیریم

۷. ما چون ۳ نفریم و زمان کمی هم داریم و ویژگی های سیستم و مهارت هامون هم فرق داره تصمیم گرفتیم که پروژه رو ۳ قسمت کنیم و به طور موازی اما با هماهنگی زیاد کار کنیم

به این صورت که مشکلات چون سیستم نسبتا قوی تری داره و تجربه و مهارت توی رابط کاربری داره قسمت رابط کاربری و تست و انتخاب مدل زبانی با اونه

و من (زینب) چون سیستم نسبتا متوسطی دارم قسمت هسته اصلی RAG (از امبدینگ چانک ها تا تولید پاسخ با مدل زبانی) با منه

و فاطمه هم چون سیستم ضعیف تری داره (رم ۴ گیگ و پردازنده core i3) قسمت استخراج و پیش پردازش و قطعه بندی متون با اونه

۸. کل پروژه ما به زبان پایتونه احتمالا چون حتی رابط کاربری هم با استریملیت زده میشه

۹. برای استخراج متن اسناد احتمالا از کتابخونه pymupdf و برای پیش پردازش و نرمال سازی متون از کتابخونه hazm استفاده میشه

۱۰. یسری داده بین ماژول های ما رد و بدل میشه و برای اینکه پروژه به تداخل نوع داده ای نخوره باید با قرارداد های داده ای نوع داده ای ثابت برای همه شون انتخاب کنیم

* که اگه بتونم یکی از قرارداد های داده ای که الان آماده ست رو برات میفرستم

۱۱. جریان داده ای کلی که بین ماژول های ما رد و بدل میشه به این صورته:

مشکات اسناد کاربر رو میفرسته برا فاطمه

فاطمه چانک ها و متادیتا رو میفرسته برا من

من به مشکلات اطلاع میدم که پردازش اسناد تموم شده سوال کاربر رو بگیر

مشکات سوال کاربر رو میفرسته برا فاطمه که پیشپردازش کنه و بفرسته برا من تا امبد کنم

من پاسخ تولید شده مدل زبانی رو میفرستم برا مشکلات تا به کاربر نشون بده

۱۲. ما همیشه گفت ۳ تا پایگاه داده داریم که به این صورته:

یه پوشه دیتا داریم که کلیه فایل ها (چه خام و چه پردازش شده) میرن توش و حالا یا با زیرپوشه ها و یا با آیدی مرتب و تفکیکیشن میکنیم

یه پایگاه داده برداری داریم که چانک ها و اندیس ها شون و متا دیتا و بردار ها شون توش ذخیره میشن (کروما دی بی) که برای هر چت یه کالکشن جدا ایجاد میکنه و چانک های مربوط به اون چت رو با ۴ قسمت ذخیره میکنه (متن چانک، آیدی چانک، بردار چانک، متادیتا (ضماره فایل، شماره صفحه و...))

درضمن اسم هر کالکشن آیدی همون چته که موقع ایجاد چت از مشکلات میگیریم

و یه پایگاه داده چت ها که اطلاعات چت ها رو برای نمایش توی رابط کاربری نگه داری میکنن و مشکلات پیاده سازیش کرده و بیشتر باهانش کار داره

و احتمالا چنتا جدول داره:

جدول چت ها و آیدی هاشون

و جدول فایل های چت ها (که ستون آیدی چت دارن و اینطوری از هم جدا میشن)

و جدول پیام های چت ها که باز از طریق ویژگی آیدی چت (یکی از کلیدای اصلی شون) از هم قابل تفکیک میشن

۱۳. چون پروژه مون ماژولاره و نیاز داریم هر تیکه رو جدا تست و دیباگ کنیم باید یه مجموعه فایل های تست داشته باشیم که هر ماژول رو جدا و هر مجموعه رو جدا و کل پروژه رو هم یکجا تست کنه

۱۴. یسری امکانات هم هست که حتما باید داشته باشیم :

نگهداری تاریخچه چت ها که کاربر هر موقع دلش خواست بتونه چت های قدیمی رو ادامه بده

امکان حذف و افزودن فایل از چت ، هر موقع که کاربر خواست

*فقط حداقل یه سند برای هر چت باید موجود باشه که چت وجود داشته باشه

و اگر کاربر هیچ فایلی بارگذاری نکرد یا همه فایلای چت رو حذف کرد با هر پیامی که میده جوابشو نمیدیم و هشدار میدیم که اول باید حداقل یه سند بارگذاری کنه یا میتونه گزینه حذف چت رو هم بزنه

امکان حذف و افزودن کل چت هم وجود داره

(مکشات باید این تغییرات فایل ها و چت ها رو به من اطلاع بده تا اطلاعات چت ها و فایل ها رو از پایگاه داده برداری هم حذف و اضافه کنم)

۱۵. یسری امکانات اختیاری و پیشنهادی هم هست که اگه وقت کردیم تهش اضافه میکنیم:

مثلا همیشه یه متغیر وضعیت داشته باشیم که مراحل پردازش اسناد و تولید پاسخ رو هرکدوم تکمیل میشه تو اون وارد کنیم و به کاربر خبر بدیم که الان سیستم تو چه وضعیتیته (مثلا در حال پیش پردازش اسناد..) که کاربر حس نکنه هیچ کاری داریم نمیکنیم و کلافه بشه

یا تاریخ زدن برای چت ها

یا مثلا آخرین کارایی که کرده تو برنامه رو بهش نمایش بدیم

مثلا بگیم تو دیروز توی این چت این سوالو پرسیدی و این فایل رو اضافه کردی

یا ...

جدید ترین تغییراتی که توی پروژه دادیم:

feat(rag) پیاده سازی و تست ماژول Embedder ، زیرساخت مدیریت مدل ها و قرارداد داده ای

توی این مرحله، زیرساخت اصلی امبدینگ (تبدیل متن به بردار) رو طبق هماهنگی ها و معماری پروژه مون به صورت کامل و تست شده بالا آوردم. تمام فایل ها کاملاً ماژولار و هماهنگ با تسک های فاطمه و مشکلات پیاده شدن.

اینم توضیح فایل های اضافه/تغییر کرده، کارکردشون و نحوه استفاده:

۱. فایل ([schemas.py](#)) قرارداد داده ای مشترک)

کارش چیه؟ کلاس DocumentChunk رو تعریف کرده. این کلاس مثل یک زبان مشترک بین ما ۳ نفره؛ یعنی فاطمه بعد از تکه تکه کردن PDF ، چانک ها رو توی این قالب می ریزه، من با همین قالب چانک ها رو امبد می کنم، و مشکلات هم برای نمایش شماره صفحه و منبع تو UI از اطلاعات همین استفاده می کنه.

فیلدهاش چیه؟ شناسه چانک (chunk_id) ، شناسه سند (doc_id) ، متن چانک (text) ، شماره صفحه مبدأ (page_number) و ایندکس شروع و پایان متن در صفحه.

۲. فایل ([config.py](#)) مرکز فرماندهی پروژه)

کارش چیه؟ تمام تنظیمات، ابعاد بردار ها توی این فایل.

چرا اینطوری زدیم؟ برای اینکه موقع توسعه روی سیستم محلی از مدل بسیار سبک e5-small (ابعاد ۳۸۴) استفاده می‌کنیم، ولی وقتی رفتیم روی کولب یا سیستم قوی‌تر، فقط با تغییر یک متغیر (ACTIVE_EMBEDDING) کل پروژه روی مدل سنگین‌تر سوئیچ می‌کنه و نیازی نیست کدهای دیگه دست بخورن!

قابلیت ویژه: اگر مدل امبدینگ تغییر کنه، خود سیستم متوجه تفاوت ابعاد بردار میشه و برای اینکه دیتابیس برداری کرش نکنه، دیتابیس قبلی رو به صورت خودکار ریست می‌کنه.

۳. فایل `model_manager.py` مدیریت و دانلود خودکار مدل‌ها)

کارش چیه؟ وقتی برنامه‌ای مدل رو لازم داره، این فایل ابتدا پوشه `models/` رو می‌گرده. اگر مدل قبلاً دانلود شده باشه، همون رو لود می‌کنه. اگر نباشه، خودش به صورت خودکار دانلودش می‌کنه و ذخیره می‌کنه.

فایده: نیازی نیست مدل‌ها رو دستی دانلود یا جابه‌جا کنیم.

۴. فایل `embedder.py` (ماژول اصلی امبدر)

کارش چیه؟ متن چانک‌های فاطمه یا سوال کاربر رو می‌گیره و تبدیل به لیست اعداد اعشاری (بردار عددی) می‌کنه.

توابع اصلی:

`embed_chunks(chunks):` از چانک‌ها رو می‌گیره و یک دیکشنری استاندارد و آماده برای ورود مستقیم به ChromaDB شامل `ids, documents, embeddings, metadatas` تحویل میده.

`embed_query(query_text):` سوال کاربر رو می‌گیره و بردارش رو برای بازیابی تحویل میده.

هوشمندی E5: به صورت خودکار برای چانک‌ها پیشوند `passage:` و برای سوال پیشوند `query:` رو اضافه می‌کنه تا دقت مدل E5 افت نکنه.

۵. فایل `tests/test_embedder.py` تست جامع امبدر)

کارش چیه؟ ۵ تا تست خودکار داره که لود مدل، ابعاد ۳۸۴، فرمت پیشوندها، سازگاری خروجی با ChromaDB و درک شباهت معنایی (تشخیص تفاوت متن مالی با مرخصی) رو چک می‌کنه.

نحوه اجرا: کافیست توی ترمینال بنویسید `python tests/test_embedder.py` :



نحوه استفاده سریع هم‌تیمی‌ها از ماژول امبدر (مثال کد):

Apply

```
from embedder import Embedder
```

```
from schemas import DocumentChunk
```

#۱. ساخت شیء امبدر (خودش مدل e5-small رو لود یا دانلود می‌کنه)

```
embedder = Embedder()
```

#۲. تبدیل چانک‌های فاطمه به بردار برای ذخیره در ChromaDB

```
chroma_data = embedder.embed_chunks(chunks_list)
```

#۳. برداری کردن سوال کاربر برای جستجو

```
query_vector = embedder.embed_query("سوال کاربر...")
```



مراحل انجام کار قبل از کامیت و پوش زدن (قدم به قدم):

گام ۱: آپدیت فایل .gitignore. فایل .gitignore. پروژه رو باز کن و این چند خط رو به انتهای فایل اضافه کن تا فایل‌های حجیم و دیتابیس‌های تست لوکال نره روی گیت‌هاب:

Apply

#فایل‌های مدل تست محلی

test_models_folder/

#دیتابیس‌های لوکال SQLite

*.db

database/*.db

#دیتابیس برداری کروما

chroma_db/

#پوشه کلون اضافی

offline-pdf-qa/

گام ۲: کامیت و پوش زدن از ترمینال) یا از رابط گیت (VS Code

اگر از ترمینال استفاده می‌کنی، دستورات زیر رو اجرا کن:

Run

git add .

git commit -m "feat(rag): Embedder، زیرساخت مدیریت مدل‌ها و قرارداد داده‌ای"

git push origin master

با اجرای این گام‌ها، تمام زحماتت برای مازول امبدر و زیرساخت‌هاش کاملاً تمیز و حرفه‌ای روی گیت‌هاب قرار می‌گیره و می‌تونیم بلافاصله بریم سراغ ساخت مازول پایگاه داده برداری. ([vector_db.py](#))

و تغییرات لازم توی گیت هاب و تغییرات غیر لازم تو یگیت ایگنور اضافه شدن